به نام خدا

طراحی کامپایلر و زبان‌های برنامه‌نویسی — بهار ۱۴۰۵

تمرین شماره ۳

آخرین مهلت تحویل: ۲۴ خرداد ۱۴۰۵

سوال ۱ : تحلیل صفت‌های گرامر

دستیار آموزشی: امیرارسلان شهبازی

فرض کنید گرامر زیر را داریم:

$$S \to AB$$
$$A \to CD$$
$$B \to EF$$

هر غیرپایانه دو صفت دارد:

s : attribute synthesized

i : attribute inherited

برای هر یک از مجموعه قواعد زیر تعیین کنید:

۱. آیا S-Attributed است؟

۲. آیا L-Attributed است؟

۳. آیا ترتیبی برای مقداردهی صفت‌ها در حین پارس وجود دارد؟

در هر بخش، دلیل انتخاب پاسخ خود را به طور مختصر بیان کنید.

الف)

$$A.s = C.s + D.s$$
$$B.s = E.s + F.s$$
$$S.s = A.s + B.s$$

ب)

$$A.i = B.s$$
$$B.i = A.s$$
$$A.s = A.i + C.s$$
$$B.s = B.i + E.s$$

ج)

$$A.i = B.s$$
$$A.s = C.s + D.s$$
$$B.s = E.s + F.s$$
$$S.s = A.s + B.s$$

د)

$$A.i = S.i$$
$$C.i = A.i$$
$$D.i = C.s + A.i$$
$$A.s = C.s + D.s$$
$$B.i = A.s + S.i$$
$$E.i = B.i$$
$$F.i = E.s + B.i$$
$$B.s = E.s + F.s$$
$$S.s = A.s + B.s$$

سوال ۲ : پیاده‌سازی الگوریتم Thompson با SDT صفت‌محور

دستیار آموزشی: امیرارسلان شهبازی

الگوریتم Thompson را در نظر بگیرید که یک عبارت منظم را به یک ماشین متناهی غیرقطعی (NFA) با transition ε تبدیل می‌کند.
با استفاده از یک L-attributed SDT، این الگوریتم را پیاده‌سازی کنید. فرض کنید توکنی به نام `char` وجود دارد که هر کاراکتر را نمایش می‌دهد و مقدار `char.val` همان کاراکتر متناظر است. همچنین تابعی به نام `new()` هر بار یک حالت جدید و یکتا برمی‌گرداند.

بخش ۱: تولید L-Attributed SDT
راه حل شما باید شامل موارد زیر باشد:

- گرامر CFG برای عبارت‌های منظم که شامل موارد زیر باشند:
  - `char`
  - `union (|)`
  - `concatenation`
  - `star Kleene (*)`
  - `parentheses`
- ویژگی‌های معنایی (attributes semantic)
- قواعد معنایی (rules semantic)

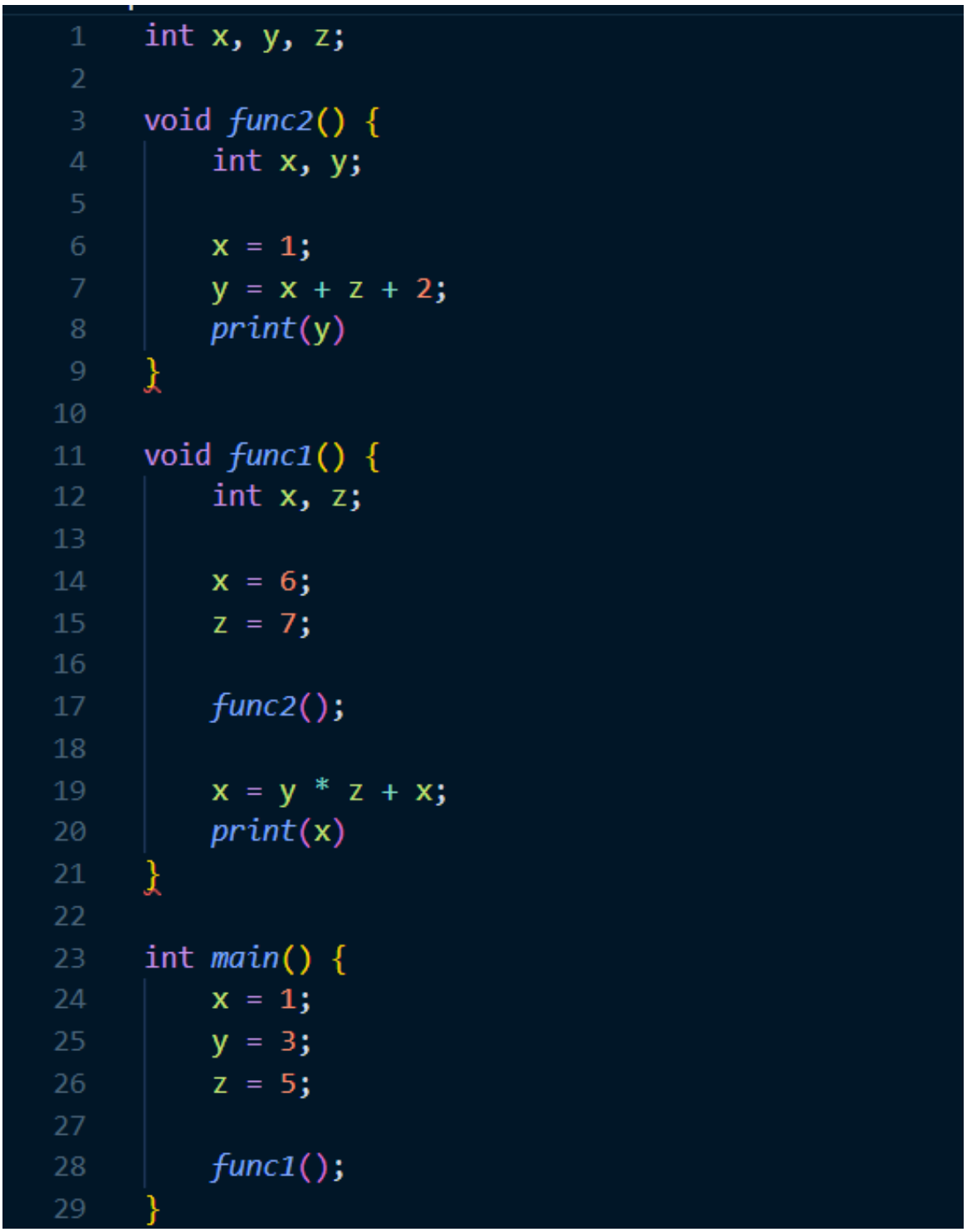
بخش ۲: کاربرد SDT
با استفاده از SDT و گرامر خود، NFA متناظر با ورودی زیر را بسازید و مراحل آن را نشان دهید:

$$a \mid (ab^*a)$$

سوال ۳: Static vs Dynamic scoping

دستیار آموزشی: فاطمه بلوک آذری

الف) خروجی کد زیر را در هر دو حالت static scoping و dynamic scoping مشخص کنید.



```
int x, y, z;

void func2() {
    int x, y;

    x = 1;
    y = x + z + 2;
    print(y)
}

void func1() {
    int x, z;

    x = 6;
    z = 7;

    func2();

    x = y * z + x;
    print(x)
}

int main() {
    x = 1;
    y = 3;
    z = 5;

    func1();
}
```

ب) Dynamic scoping امکان استفاده منعطف‌تری به ما می‌دهد. با این وجود، چرا همواره از dynamic scoping استفاده نمی‌کنیم؟

سوال ۴: تحلیل نام و درخت استنتاج

دستیار آموزشی: فاطمه بلوک آذری

برای خط مشخص شده ابتدا Γ را بدست آورید و سپس با استفاده از درخت استنتاج بررسی کنید که این خط از نظر تحلیل‌گر syntax نامعتبر است یا خیر.

```
class Base {
    int p;
    int q;

    Base(int p, int q) {
        this.p = p;
        this.q = q;
    }

    int compute(int t) {
        return p + q + t;
    }
}

class Derived extends Base {
    Derived(int p, int q)
    {
        super(p, q);
    }

    float validate(int limit) {
        Base obj = new Base(10, 5);
        int val = 20;

        if (val == obj.compute(val) && this.compute(obj.p) > 0) { // Name Analyze condition of if
            return 1.0f;
        }
        else if (check(val, limit, obj)) {
            return 2.0f;
        }
        return 0.0f;
    }

    boolean check(int a, int b, Base ref) {
        return ref.compute(0) > b ? (a > ref.p) : (this.compute(0) == a);
    }
}
```

سوال ۵: تحلیل نام دستور let در Kotlin

دستیار آموزشی: فاطمه بلوک آذری

در زبان Kotlin، دستور `let` یک scope جدید ایجاد می‌کند که از طریق آن می‌توان به فیلدها و متدهای یک `object` دسترسی داشت و مقداری به آن‌ها نسبت داد.
دقت کنید که `it` در اینجا مشابه `this` در Java و C++ عمل می‌کند.
برای این دستور قواعد تحلیل نام بنویسید سپس با استفاده از قواعدی که نوشتید، درخت استنتاج را برای مثال زیر رسم کنید

```
fun main() {

    data class Person(var name: String, var tutorial: String)

    var person = Person("Anupam", "Kotlin")

    person.let {
        it.name = "No Name"
        it.tutorial = "Advanced Kotlin"
    }

    println(person)
}
```